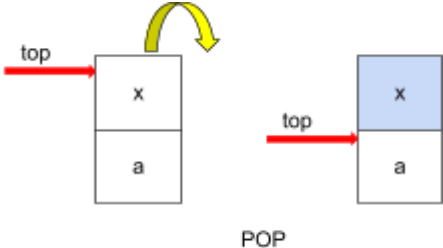
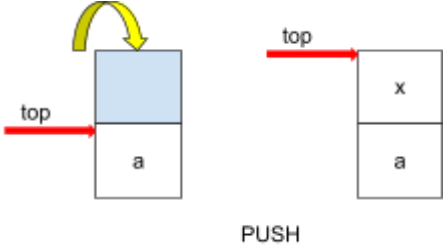


ESTRUCTURA DE DATOS

PILA (LIFO) CON ARREGLOS

```
1  import java.util.EmptyStackException;
2
3  public class ArrayPila2{
4
5      // Atributos
6      private int[] array;          // Array de enteros
7      private int topOfStack;       // Índice del tope de la pila
8      private static final int TAM = 10; // Capacidad fija
9
10     // Constructor
11     public ArrayPila2() {
12         array = new int[TAM];
13         topOfStack = -1; // Pila vacía
14     }
15
16     // Inserta un elemento en el tope
17     public void push(int x) {
18         if (topOfStack + 1 == array.length) {
19             System.out.println(x:"Error: Pila llena. No se puede insertar más elementos.");
20             return;
21         }
22         array[++topOfStack] = x;
23     }
24
25     // Elimina el elemento del tope
26     public void pop() throws EmptyStackException {
27         if (isEmpty()) {
28             throw new EmptyStackException();
29         }
30         topOfStack--;
31     }
32
33     // Devuelve el tope sin eliminar
34     public int top() throws EmptyStackException {
35         if (isEmpty()) {
36             throw new EmptyStackException();
37         }
38         return array[topOfStack];
39     }
40
41     // Devuelve y elimina el tope
42     public int topAndPop() throws EmptyStackException {
43         if (isEmpty()) {
44             throw new EmptyStackException();
45         }
46         return array[topOfStack--];
47     }
48
49     // ¿Está vacía?
50     public boolean isEmpty() {
51         return topOfStack == -1;
52     }
53
54     // Vacía la pila
55     public void makeEmpty() {
56         topOfStack = -1;
57     }
58
59     // Imprime toda la pila
60     public void imprimirPilaCompleta() {
61         if (isEmpty()) {
62             System.out.println(x:"La pila está vacía.");
63         } else {
64             System.out.println(x:"Contenido de la pila:");
65             for (int i = 0; i <= topOfStack; i++) {
66                 System.out.println("[ " + i + " ] : " + array[i]);
67             }
68         }
69     }
70 }
```



COLA (FIFO) CON ARREGLOS

```

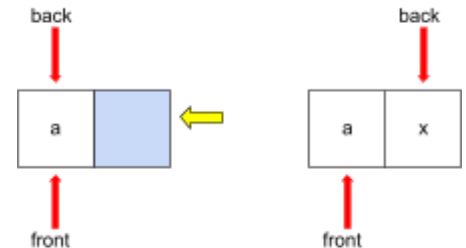
1  import java.util.NoSuchElementException;
2
3  public class ArrayColas2 {
4
5      // Atributos
6      private static final int TAM = 10;
7      private int[] array;
8      private int front;           // Índice del frente
9      private int back;           // Índice del final
10     private int currentSize;     // Cantidad actual de elementos
11
12     // Constructor
13     public ArrayColas2() {
14         array = new int[TAM];
15         makeEmpty();
16     }
17
18     // Incrementa el índice circularmente
19     private int incremento(int x) {
20         return (++x == array.length) ? 0 : x;
21     }
22
23     // Duplica el arreglo si se llena
24     private void doubleCola() {
25         int[] nuevoArray = new int[array.length * 2];
26         for (int i = 0; i < currentSize; i++, front = incremento(front)) {
27             nuevoArray[i] = array[front];
28         }
29         array = nuevoArray;
30         front = 0;
31         back = currentSize - 1;
32     }
33
34     // Agrega un elemento a la cola
35     public void enqueue(int x) {
36         if (currentSize == array.length) {
37             System.out.println(x: "La cola está llena. Se redimensiona.");
38             doubleCola();
39         }
40         back = incremento(back);
41         array[back] = x;
42         currentSize++;
43     }
44
45     // Devuelve el elemento en el frente sin eliminarlo
46     public int getFront() {
47         if (isEmpty()) {
48             throw new NoSuchElementException(s: "Cola vacía");
49         }
50         return array[front];
51     }
52
53     // Elimina y devuelve el primer elemento (FIFO)
54     public int dequeue() {
55         if (isEmpty()) {
56             throw new NoSuchElementException(s: "Cola vacía");
57         }
58         int valor = array[front];
59         front = incremento(front);
60         currentSize--;
61         return valor;
62     }
63
64     // Verifica si está vacía
65     public boolean isEmpty() {
66         return currentSize == 0;
67     }
68
69     // Vacía la cola
70     public void makeEmpty() {
71         currentSize = 0;
72         front = 0;
73         back = -1;
74     }
75
76     // Imprime el contenido de la cola
77     public void imprimirCola() {
78         if (isEmpty()) {
79             System.out.println(x: "Cola vacía.");
80             return;
81         }
82
83         System.out.println(x: "Contenido de la cola:");
84         for (int i = 0; i < currentSize; i++) {
85             int idx = (front + i) % array.length;
86             System.out.println("[" + idx + "]: " + array[idx]);
87         }
88     }
89 }

```

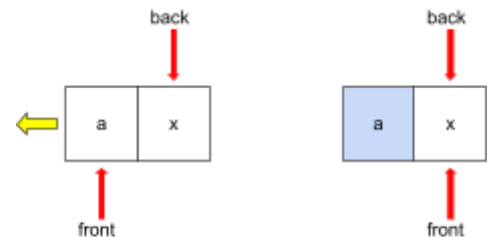
```

private int incremento(int x){
    if(++x == array.length){
        x=0;
    }
    return x;
}

```



ENQUEUE



DEQUEUE

LISTAS ENLAZADAS SIMPLES

(La clase nodo es común a todo lo relacionado con listas enlazadas, incluyendo pila y cola con listas enlazadas)

```
1  class Nodo {
2  //atributos
3      Object data; // Datos del nodo
4      Nodo next; // Referencia al siguiente nodo
5  //constructor vacio
6
7
8  // Constructor que inicializa el nodo con dato y el siguiente nodo
9  public Nodo (Object d, Nodo n){
10     data = d;
11     next = n;
12 }
13
14 public Nodo() {
15 }
16
17 // Constructor que solo inicializa el dato, el siguiente nodo se establece como null
18 public Nodo(Object d) {
19     this(d, null);
20 }
21 //getters y setters
22 public void setData(Object data) {
23     this.data = data;
24 }
25
26 public void setNext(Nodo next) {
27     this.next = next;
28 }
29
30 public Object getData() {
31     return data;
32 }
33
34 public Nodo getNext() {
35     return next;
36 }
37
38 }
```

```

1 public class ListaEnlazada {
2     //atributos
3     private Nodo head;
4     private Nodo current;
5
6     //constructor
7
8     public ListaEnlazada(Nodo head, Nodo current) {
9         this.head = head;
10        this.current = current;
11    }
12
13    public ListaEnlazada() {
14    }
15
16    //metodo para agregar un nodo a la lista
17    public void agregarNodo (Object data){
18        Nodo nuevoNodo = new Nodo (data);
19
20        if(head == null){
21            head = nuevoNodo;
22            current = nuevoNodo;
23        }
24        else{
25            Nodo temp = new Nodo(data, current.next);
26            current.next = temp;
27        }
28    }
29
30    //metodo para borrar un nodo
31    public void borrarNodo (Object data){
32        // Caso cuando la lista está vacía
33        if (head == null) {
34            return; // No hay nada que borrar
35        }
36
37        // Caso cuando el nodo a borrar es la cabeza
38        if (head.data.equals(data)) {
39            head = head.next; // Mueve la cabeza al siguiente nodo
40            return; // Sale del método
41        }
42        current = head;
43        while ((current.next != null)) {
44            if(current.next.data.equals(data)){
45                current.next = current.next.next;
46                return;
47            }
48            current=current.next;
49        }
50    }
51
52    // Método para recorrer la lista y mostrar los datos de cada nodo
53    public void recorrer() {
54        Nodo actual = head; // Comienza en la cabeza
55        int indice = 0; // Para mostrar el índice de cada nodo
56        while (actual != null) { // Mientras no se llegue al final de la lista
57            System.out.println "[" + indice + "]: " + actual.data); // Imprime el índice y el valor
58            actual = actual.next; // Avanza al siguiente nodo
59            indice++; // Incrementa el índice
60        }
61    }
62
63    //Mover el elemento ingresado al principio de la lista
64    public void mover (Object data){
65        Nodo nuevonodo = new Nodo (data);
66        nuevonodo.next=head;
67        head = nuevonodo;
68    }
69 }

```

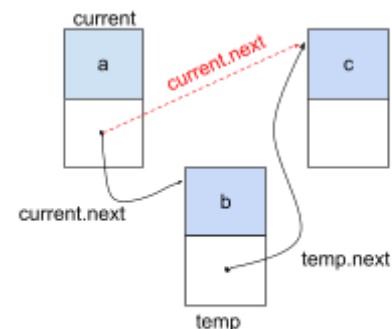
En caso de no haber creado el constructor en la clase nodo:

```

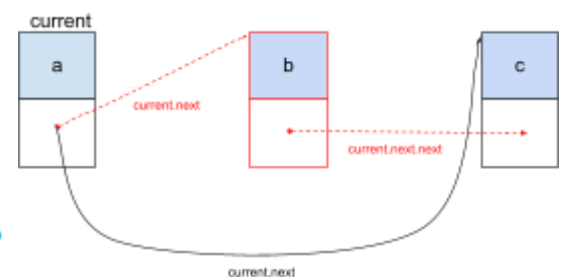
//metodo para agregar un nodo a la lista
public void agregarNodo (Object data){

    Nodo temp = new Nodo (data);
    temp.data = data;
    temp.next = current.next;
    current.next = temp;
}

```



Agregar Nodo



Eliminar Nodo

PILA (LIFO) CON LISTAS ENLAZADAS

```
1  import java.util.EmptyStackException;
2
3  public class ListaPila {
4
5      private Nodo top; // Apunta al tope de la pila
6
7      public ListaPila() {
8          top = null;
9      }
10
11      // Inserta un elemento en la pila
12      public void push(Object x) {
13          Nodo nuevo = new Nodo(x, top);
14          top = nuevo;
15      }
16
17      // Elimina el elemento en el tope de la pila
18      public void pop() throws EmptyStackException {
19          if (isEmpty()) {
20              throw new EmptyStackException();
21          }
22          top = top.next;
23          System.out.println(x: "\nSe ha borrado el elemento del tope");
24      }
25
26      // Devuelve el elemento en el tope sin eliminarlo
27      public Object top() throws EmptyStackException {
28          if (isEmpty()) {
29              throw new EmptyStackException();
30          }
31          return top.data;
32      }
33
34      // Elimina y devuelve el elemento del tope
35      public Object topAndPop() throws EmptyStackException {
36          Object valor = top.data;
37          pop();
38          return valor;
39      }
40
41      // Verifica si está vacía
42      public boolean isEmpty() {
43          return top == null;
44      }
45
46      // Vacía la pila
47      public void makeEmpty() {
48          top = null;
49      }
50
51      // Imprime el contenido de la pila
52      public void imprimirPilaCompleta() {
53          if (isEmpty()) {
54              System.out.println(x: "La pila está vacía.");
55              return;
56          }
57
58          Nodo actual = top;
59          int i = 0;
60          System.out.println(x: "Contenido de la pila:");
61          while (actual != null) {
62              System.out.println("[" + i + "] : " + actual.data);
63              actual = actual.next;
64              i++;
65          }
66      }
67  }
```

COLA (FIFO) CON LISTAS ENLAZADAS

```
1  import java.util.NoSuchElementException;
2
3  public class ListaCola {
4
5      private Nodo front; // Frente de la cola
6      private Nodo back;  // Final de la cola
7      private int currentSize;
8
9      public ListaCola() {
10         makeEmpty();
11     }
12
13     // Agrega un elemento al final
14     public void enqueue(Object x) {
15         Nodo nuevo = new Nodo(x);
16
17         if (isEmpty()) {
18             front = nuevo;
19             back = nuevo;
20         } else {
21             back.next = nuevo;
22             back = nuevo;
23         }
24
25         currentSize++;
26     }
27
28     // Devuelve el frente sin eliminarlo
29     public Object getFront() {
30         if (isEmpty()) {
31             throw new NoSuchElementException(s:"Cola vacía");
32         }
33         return front.data;
34     }
35
36     // Elimina y devuelve el frente (FIFO)
37     public Object dequeue() {
38         if (isEmpty()) {
39             throw new NoSuchElementException(s:"Cola vacía");
40         }
41         Object valor = front.data;
42         front = front.next;
43         currentSize--;
44
45         if (isEmpty()) {
46             back = null; // por si queda vacía
47         }
48
49         return valor;
50     }
51
52     // Verifica si está vacía
53     public boolean isEmpty() {
54         return currentSize == 0;
55     }
56
57     // Vacía la cola
58     public void makeEmpty() {
59         front = null;
60         back = null;
61         currentSize = 0;
62     }
63
64     // Imprime el contenido de la cola
65     public void imprimirCola() {
66         if (isEmpty()) {
67             System.out.println(x:"Cola vacía.");
68             return;
69         }
70
71         Nodo actual = front;
72         int i = 0;
73         System.out.println(x:"Contenido de la cola:");
74         while (actual != null) {
75             System.out.println("[" + i + "]: " + actual.data);
76             actual = actual.next;
77             i++;
78         }
79     }
80 }
```

ORDENAMIENTO

INSERCIÓN

```
1 public class Insercion {
2     public static void main(String[] args) {
3         int[] arreglo = {5, 2, 9, 1, 5, 6}; // Arreglo desordenado
4
5         System.out.println(x:"Arreglo antes de ordenar:");
6         imprimirArreglo(arreglo);
7
8         // Llamamos al método de ordenamiento por inserción
9         ordenamientoInsercion(arreglo);
10
11        System.out.println(x:"Arreglo ordenado de menor a mayor:");
12        imprimirArreglo(arreglo);
13    }
14
15    // Método de ordenamiento por inserción
16    public static void ordenamientoInsercion(int[] arreglo) {
17        // 1. Bucle externo recorre todo el arreglo a partir del segundo elemento
18        for (int i = 1; i < arreglo.length; i++) {
19            int j=i;
20
21            // 2. Bucle interno compara el elemento actual con los anteriores y los desplaza
22            while ( arreglo[j] < arreglo[j-1] ) {
23                // 3. Intercambio: Se desplaza el elemento mayor hacia la derecha
24                int aux = arreglo [j-1];
25                arreglo [j-1] = arreglo [j];
26                arreglo[j] = aux;
27                // 4. Decrementa j para seguir comparando hacia la izquierda
28                j--;
29            }
30        }
31    }
32
33    // Método para imprimir el arreglo
34    public static void imprimirArreglo(int[] arreglo) {
35        for (int num : arreglo) {
36            System.out.print(num + " ");
37        }
38        System.out.println();
39    }
40 }
```

Vector original

45	17	23	67	21
----	----	----	----	----

Iteración 1:

17 < 45? ☒	45	17	23	67	21	Se genera cambio
23 < 45? ☒	17	45	23	67	21	Se genera cambio
67 < 45? ☐	17	23	45	67	21	No hay cambio
21 < 67? ☒	17	23	45	67	21	Se genera cambio
	17	23	45	21	67	Fin primera iteración

QUICKSORT

```

1  public class QuickSort {
    Run | Debug
2  public static void main(String[] args) {
3      int[] arreglo = {22, 7, 2, 15, 18, 10, 25, 1, 30};
4      System.out.println(x:"Arreglo antes de ordenar:");
5      imprimirArreglo(arreglo);
6
7      quickSort(arreglo, left:0, arreglo.length - 1);
8
9      System.out.println(x:"\nArreglo ordenado:");
10     imprimirArreglo(arreglo);
11 }
12
13 // Método para imprimir el arreglo
14 static void imprimirArreglo(int[] arreglo) {
15     for (int num : arreglo) {
16         System.out.print(num + " ");
17     }
18     System.out.println();
19 }
20 // Método QuickSort
21 static void quickSort(int[] arreglo, int left, int right) {
22     int i, j, temp;
23     i=left; //índice izquierdo
24     j=right; //índice derecho
25     do{
26         // Mueve j hacia la izquierda hasta encontrar un elemento menor o igual que el pivote
27         while (arreglo[j] > arreglo[i] && j>i)j--;
28         if(i < j){
29             temp=arreglo[i];
30             arreglo[i]=arreglo[j];
31             arreglo[j]=temp;
32             i++;
33         }
34         // Mueve i hacia la derecha hasta encontrar un elemento mayor o igual que el pivote
35         while (arreglo[i] < arreglo[j] && i<j)i++;
36         if(i<j){
37             temp=arreglo[i];
38             arreglo[i]=arreglo[j];
39             arreglo[j]=temp;
40             j--;
41         }
42     }while(i<j);
43     // Ordena la sublista a la izquierda del pivote
44     if(left < j) quickSort(arreglo, left, j-1);
45     // Ordena la sublista a la derecha del pivote
46     if(i<right) quickSort(arreglo, i+1, right);
47 }
48 }

```

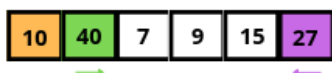
1º ELEMENTO=VALOR PIVOTE



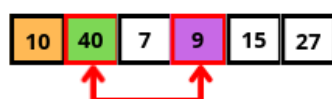
BÚSQUEDA DE IZQUIERDA A DERECHA
i > VALOR PIVOTE



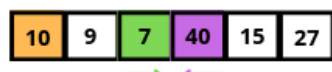
BÚSQUEDA DE IZQUIERDA A DERECHA
j < VALOR PIVOTE



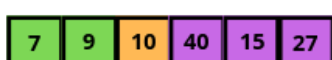
COMIENZA LA BÚSQUEDA



40 > 10
9 < 10 → INTERCAMBIAMOS AMBOS
NÚMEROS



SE SIGUE BUSCANDO, PERO DEBIDO A QUE NO HAY NUMERO MAYOR A 40 A LA IZQUIERDA
TIENE QUE COMPARARSE CONSIGO MISMO, INDICIO DE QUE ESTÁ EN SU DESTINO FINAL
OBTENEMOS QUE 7 < 10 Y 40 > 10 Y SE INTERCAMBIA EL MENOR CON EL PIVOTE



SUBARRAY 1

SUBARRAY 2

OBTENEMOS DOS SUBARRAYS A LOS QUE SE LES VA A APLICAR EL MISMO PROCESO

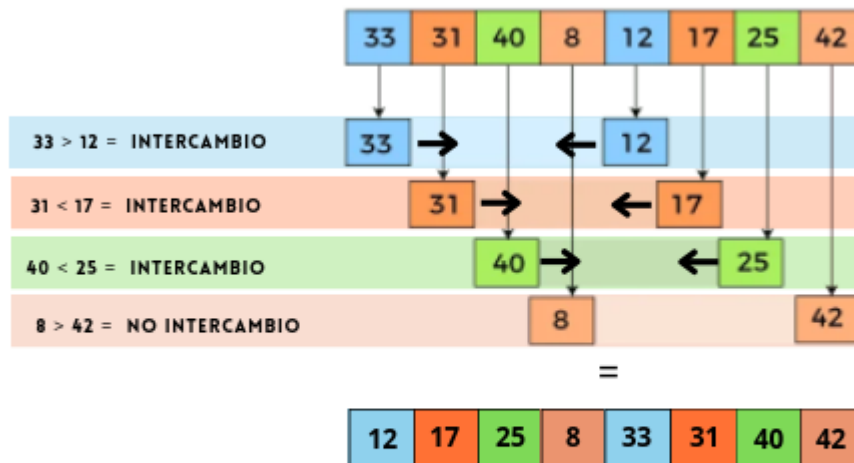
SHELLSORT

```

1  public class Shell {
2      public static void ordenamientoShell(int[] arreglo) {
3          int n = arreglo.length;
4          // 1. Ciclo principal que controla el valor del gap (brecha)
5          for (int gap = n / 2; gap > 0; gap /= 2) {
6              // 2. Recorremos los elementos comenzando desde el índice gap
7              for (int i = gap; i < n; i++) {
8                  // 3. Ciclo interno que compara y mueve los elementos según el valor del gap
9                  for (int j = i - gap; j >= 0 && arreglo[j] > arreglo[j + gap]; j -= gap) {
10                     int temp = arreglo[j];
11                     arreglo[j] = arreglo[j + gap];
12                     arreglo[j + gap] = temp;
13                 }
14             }
15         }
16     }
17
18     public static void main(String[] args) {
19         int[] arreglo = {12, 34, 54, 2, 3};
20         System.out.println("Arreglo antes de ordenar:");
21         for (int num : arreglo) {
22             System.out.print(num + " ");
23         }
24         System.out.println();
25
26         // Llamada al método de Shell Sort
27         ordenamientoShell(arreglo);
28
29         System.out.println("Arreglo después de ordenar:");
30         for (int num : arreglo) {
31             System.out.print(num + " ");
32         }
33     }
34 }

```

1 RECORRIDO - INTERVALO = 4 (8/2)



2 RECORRIDO - INTERVALO = 2 (4/2)

⋮

EL PROCESO SE REPITE HASTA QUE LOS INTERVALOS SEAN DE 1 Y COMIENCE A COMPORTARSE COMO EL ALGORITMO DE INSERCIÓN

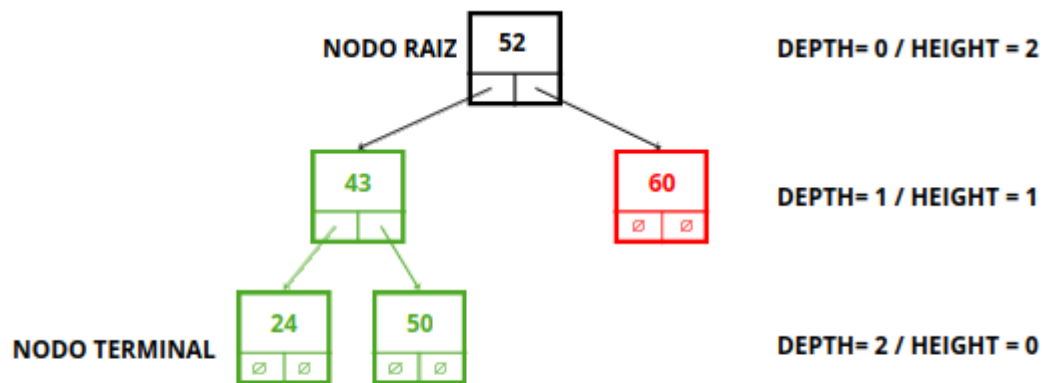
ÁRBOLES

Característica	BST	AVL	Rojinegro
Balanceo	✗ No garantiza	✓ Siempre	⚠ Parcial (relajado)
Complejidad (ops)	$O(n)$ en el peor caso	$O(\log n)$ garantizado	$O(\log n)$ garantizado
Inserción/Eliminación	Rápida pero puede fallar	Lenta por rotaciones	Más rápida que AVL
Implementación	Fácil	Intermedia	Compleja

BINARIO DE BÚSQUEDA

Es un árbol binario donde **por cada nodo**:

- Todos los valores del **subárbol izquierdo** son **menores**.
- Todos los valores del **subárbol derecho** son **mayores**.



DEPTH (Profundidad) : Distancia desde el nodo en el que estoy al nodo raíz

HEIGHT (Altura) : Distancia desde el nodo en el que estoy al nodo terminal

OPERACIONES BÁSICAS:

- **INSERCIÓN:**

Debe compararse el valor o (clave del elemento) a insertar con la raíz del árbol. Si es mayor, debe avanzar hacia el subárbol derecho. Si es menor, debe avanzar hacia el subárbol izquierdo.

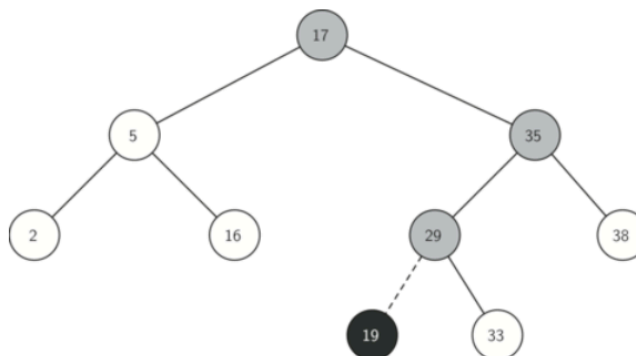
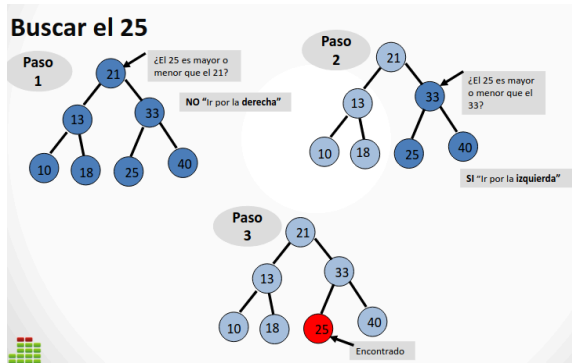


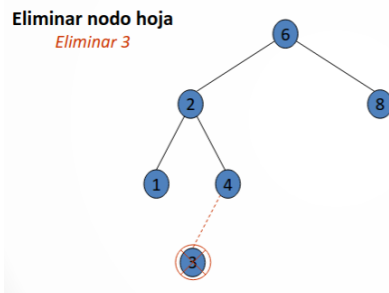
Figura 2: Inserción de un nodo con clave = 19

- BÚSQUEDA:

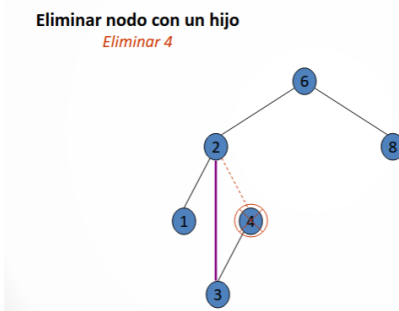


- ELIMINACIÓN:

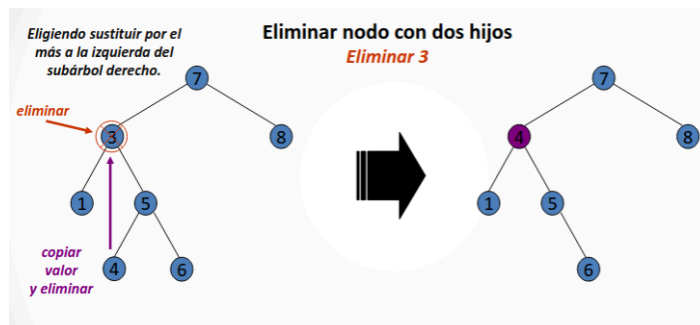
- Si el elemento a borrar es terminal (hoja)



- Si el elemento a borrar tiene un solo hijo



- Si el elemento a borrar tiene los dos hijos
Se tienen que sustituir por el nodo que se encuentra más a la derecha en el subárbol izquierdo



- MÉTODOS PARA RECORRER EL ÁRBOL:

- In-orden: Izquierdo → Nodo actual → Derecho
- Pre-orden: Nodo actual → Izquierdo → Derecho
- Post-orden: Izquierdo → Derecho → Nodo actual

```

1 public class Tree {
2     //atributos
3     private Node root;
4
5     //constructor
6     public Tree() {
7         root = null;
8     }
9
10    //metodo para insertar un nodo (no permite duplicados)
11    Node insert(int value, Node t){
12        if (t == null){
13            // Si el árbol está vacío, crea un nuevo nodo con el value
14            t = new Node(value);
15        }
16        }else if(value < t.element){
17            t.left=insert(value, t.left);
18        }
19        }else if(value > t.element){
20            t.right=insert(value, t.right);
21        }
22        return t;
23    }
24    // Método para insertar un nodo (permite duplicados)
25    Node insert_duplicate(int value, Node t) {
26        if (t == null) {
27            // Si el árbol está vacío, crea un nuevo nodo con el value
28            t = new Node(value);
29        }
30        } else if (value < t.element) {
31            // Si el value es menor, se inserta en el subárbol izquierdo
32            t.left = insert_duplicate(value, t.left);
33        }
34        } else {
35            // Si el value es mayor o igual, se inserta en el subárbol derecho (permitiendo duplicados)
36            t.right = insert_duplicate(value, t.right);
37        }
38        return t; // Retorna la raíz del árbol modificado
39    }
40
41    //Método para recorrer el árbol en inorden (izquierda, raíz, derecha)
42    void inOrden(Node t){
43        if( t != null){
44            inOrden(t.left);
45            System.out.print(t.element + " ");
46            inOrden(t.right);
47        }
48    }
49
50    int depth(Node t) {
51        if (t == null) { //caso base
52            return 0; // Si el árbol está vacío, la profundidad es 0
53        }
54        } else {
55            // Llamado recursivo
56            if (depth(t.left)>depth(t.right)) {
57                return depth(t.left)+ 1; // Si el izquierdo es más profundo, sumamos 1
58            }
59            } else {
60                return depth(t.right) + 1; // Si el derecho es más profundo o igual, sumamos 1
61            }
62        }
63    }
64
65    //ELIMINAR UN NODO
66    // Método para encontrar el nodo mínimo (para el caso de dos hijos)
67    Node findMin(Node t) {
68        if(t != null){
69            while (t.left != null) {
70                t = t.left;
71            }
72        }
73        return t;
74    }
75
76    //Método para eliminar el nodo mínimo en el subárbol dado
77    //El mínimo siempre esta a la izquierda del arbol
78    Node eliminateMin(Node t){
79        if(t==null){
80            System.out.println("Arbol vacio.");
81            return t;
82        }
83        } else if(t.left != null){
84            t.left = eliminateMin(t.left);
85        }
86        } else{
87            return t.right;
88        }
89    }
90
91    Node dtree(Node root, int value) {
92        if (root == null) { //caso base: Si el arbol esta vacio
93            System.out.println("El value " + value + " no se encuentra en el árbol.");
94            return root;
95        }
96        //caso recursivo
97        // Busca el nodo a eliminar
98        if (value < root.element) {
99            root.left = dtree(root.left, value); // Buscar en el subárbol izquierdo
100        }
101        } else if (value > root.element) {
102            root.right = dtree(root.right, value); // Buscar en el subárbol derecho
103        }
104        //Nodo encontrado
105        //caso 1:Nodo con 2 hijos
106        } else if (root.left != null && root.right != null){
107            root.element = findMin(root.right).element; // Reemplaza el value por el mínimo del subárbol derecho
108            root.right = eliminateMin(root.right); // Elimina el nodo mínimo del subárbol derecho
109        }
110        } else{
111            //Caso 2:Nodo con uno o ningun hijo
112            root = (root.left != null) ? root.left : root.right; // Reemplaza el nodo por su único hijo o null
113        }
114        return root;
115    }
116
117 }

```

```

1 public class Node {
2     //atributos
3     int element;
4     Node left;
5     Node right;
6
7     //constructor
8     public Node(int e) {
9         this.element = e;
10        left = right = null;
11    }
12 }

```

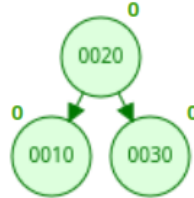
AVL

[PAGINA](#)

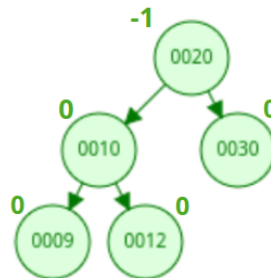
Es un árbol binario de búsqueda que satisface la condición de estar balanceado, es decir, la diferencia de altura entre el subárbol izquierdo y el subárbol derecho de cada nodo es como máximo 1.

El peso del nodo puede ser:

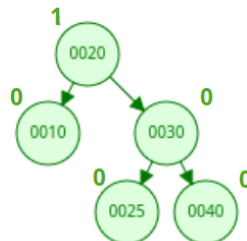
- 0 : el subárbol izquierdo y el derecho están a la misma altura



- -1 : Tiene un nivel de más a la izquierda = derecho es más alto



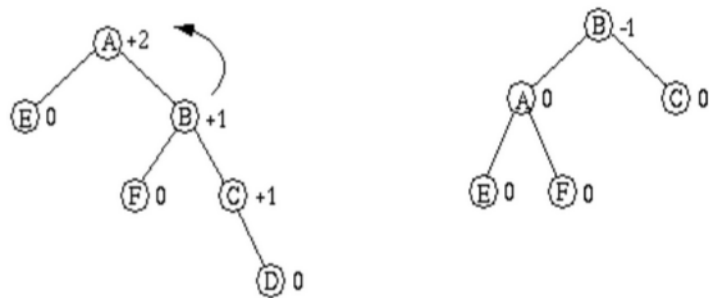
- 1 : Tiene un nivel más a la derecha = izquierdo es más alto



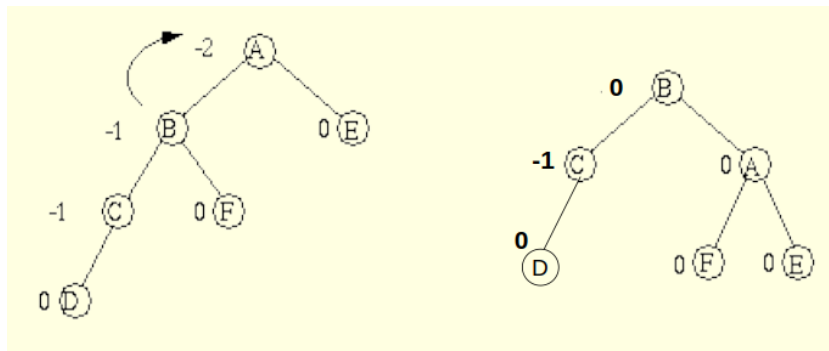
Para cada nodo, se calcula la diferencia de altura entre el subárbol izquierdo y el subárbol derecho ($\text{altura}(\text{derecho}) - \text{altura}(\text{izquierdo})$). Si esta diferencia es mayor que 1 o menor que -1, el árbol no cumple con la propiedad de balance y se debe realizar una rotación.

Rotaciones:

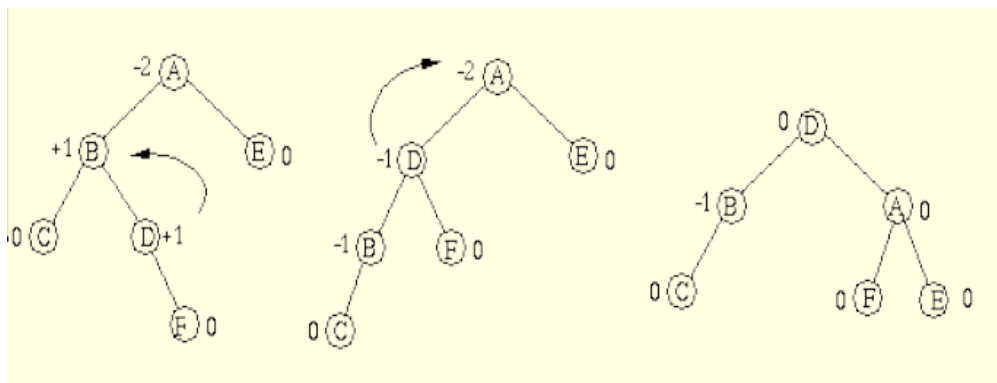
- Rotación Izquierda Simple: Desequilibrado a la izquierda y su hijo derecho tiene el mismo signo (+)



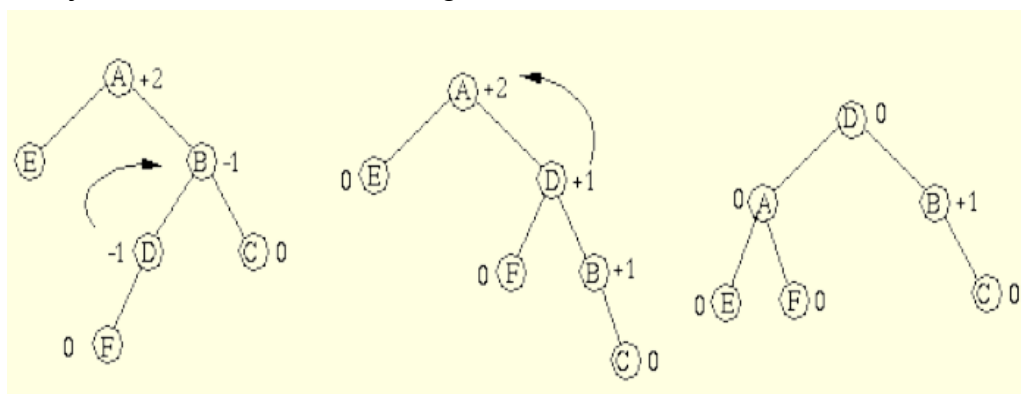
- Rotación Derecho Simple: Desequilibrio a la derecha y su hijo izquierdo tiene el mismo signo (-)



- Rotación Izquierda Doble (Izquierda-Derecha): Desequilibrado a la derecha y su hijo izquierdo tiene distinto signos



- Rotación Derecha Doble (Derecha-Izquierda): Desequilibrado a la izquierda y su hijo derecho tiene distinto signos



```

2 class ArbolAVL {
3     Nodo raiz;
4
5     // Inserta un valor en el árbol y mantiene el balance
6     public Nodo insertar(Nodo nodo, int valor) {
7
8         if (nodo == null)
9             return new Nodo(valor);
10
11         // Paso 1: Insertar como en un árbol binario de búsqueda n
12         if (valor < nodo.valor)
13             nodo.izquierdo = insertar(nodo.izquierdo, valor);
14         else if (valor > nodo.valor)
15             nodo.derecho = insertar(nodo.derecho, valor);
16         else
17             return nodo; // No se permiten duplicados
18
19         // Paso 2: Actualizar altura
20         nodo.altura = 1 + Math.max(altura(nodo.izquierdo), altura(nodo.derecho));
21
22         // Paso 3: Calcular el balance del nodo
23         int balance = altura(nodo.izquierdo) - altura(nodo.derecho);
24
25         // Rotaciones
26         //subarbol izquierdo más alto (+)
27         if (balance > 1 && valor < nodo.izquierdo.valor)
28             return rotarDerecha(nodo); // Left Left
29
30         if (balance > 1 && valor > nodo.izquierdo.valor) {
31             nodo.izquierdo = rotarIzquierda(nodo.izquierdo);
32             return rotarDerecha(nodo); // Left Right
33         }
34
35         //subarbol derecho más alto (-)
36         if (balance < -1 && valor > nodo.derecho.valor)
37             return rotarIzquierda(nodo); // Right Right
38
39         if (balance < -1 && valor < nodo.derecho.valor) {
40             nodo.derecho = rotarDerecha(nodo.derecho);
41             return rotarIzquierda(nodo); // Right Left
42         }
43
44         return nodo; // Sin rotación necesaria
45     }
46
47     //obtener balance del nodo
48     int getBalance(Nodo n) {
49         return (n == null) ? 0 : altura(n.izquierdo) - altura(n.derecho);
50     }
51
52     // Rotación Derecha Simple
53     private Nodo rotarDerecha(Nodo y) {
54         Nodo x = y.izquierdo;
55         Nodo T2 = x.derecho;
56
57         x.derecho = y;
58         y.izquierdo = T2;
59
60         y.altura = 1 + Math.max(altura(y.izquierdo), altura(y.derecho));
61         x.altura = 1 + Math.max(altura(x.izquierdo), altura(x.derecho));
62
63         return x;
64     }
65
66     // Rotación Izquierda Simple
67     private Nodo rotarIzquierda(Nodo x) {
68         Nodo y = x.derecho;
69         Nodo T2 = y.izquierdo;
70
71         y.izquierdo = x;
72         x.derecho = T2;
73
74         x.altura = 1 + Math.max(altura(x.izquierdo), altura(x.derecho));
75         y.altura = 1 + Math.max(altura(y.izquierdo), altura(y.derecho));
76
77         return y;
78     }
79
80     // Obtener altura del nodo
81     private int altura(Nodo nodo) {
82         return (nodo == null) ? -1 : nodo.altura;
83     }
84 }

```

```

1 package AVL;
2
3 class Nodo {
4     int valor, altura; // valor = dato del nodo
5     // altura = se usa para calcular el balance
6     Nodo izquierdo, derecho; // referencias a los subárboles
7
8     public Nodo(int valor) {
9         this.valor = valor; // asignamos el valor
10        this.altura = 0;
11        this.izquierdo = null;
12        this.derecho = null;
13    }
14 }

```

ROJINEGRO

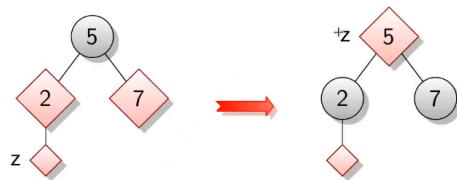
Un árbol rojo-negro es un árbol binario de búsqueda en el que cada nodo almacena un bit adicional de información llamado color, el cual puede ser rojo o negro.

PROPIEDADES:

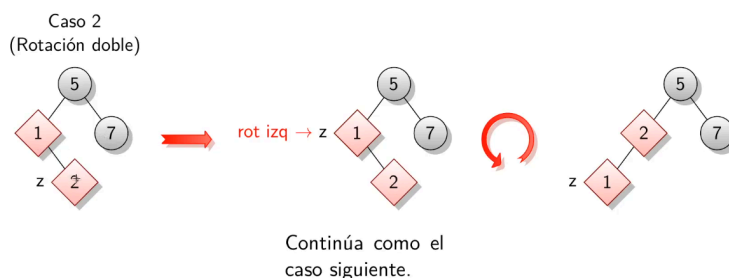
1. Propiedad de color: Cada nodo es rojo o negro.
2. Propiedad de raíz: La raíz es negra.
3. Propiedad de hojas nulas: Todas las hojas nulas (NIL) son negras.
4. Propiedad de nodos rojos: Si un nodo es rojo, ambos hijos son negros.
5. Propiedad de nodos negros: Para cada nodo, cualquier camino simple desde este nodo hasta cualquiera de sus nodos hoja descendientes tiene el mismo número de nodos negros.

Insertar un nodo:

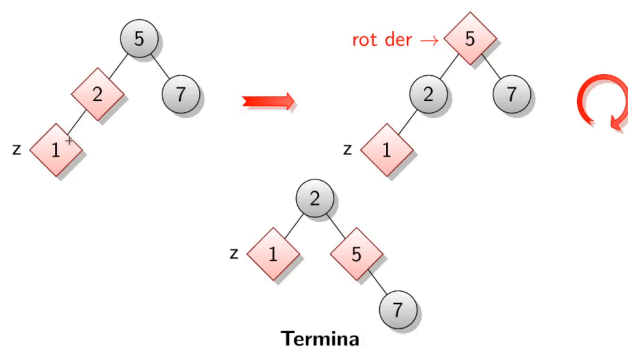
Caso 1: El padre y el tío del nodo nuevo son rojos.



Caso 2: Padre rojo, tío negro y z hijo derecho.



Caso 3: Padre rojo, tío negro y z hijo izquierdo.




```

3 public class ArbolRojinegro {
4
5     private Nodo root;
6     private Nodo tnull; // Nodo nulo (representa hojas nulas)
7
8     // Constructor
9     public ArbolRojinegro () {
10         tnull = new Nodo();
11         tnull.color = 0; // Color del Nodo nulo es negro
12         root = tnull;
13     }
14
15     // Insertar un elemento en el árbol
16     public void insertar(int key) {
17         Nodo nodo = new Nodo();
18         nodo.father = null;
19         nodo.data = key;
20         nodo.left = tnull;
21         nodo.right = tnull;
22         nodo.color = 1; // Nuevo Nodo siempre es rojo
23
24         Nodo y = null;
25         Nodo x = this.root;
26
27         while (x != tnull) {
28             y = x;
29             if (Nodo.data < x.data) {
30                 x = x.left;
31             } else {
32                 x = x.right;
33             }
34         }
35
36         // y es el padre de x
37         nodo.father = y;
38         if (y == null) {
39             root = nodo;
40         } else if (Nodo.data < y.data) {
41             y.left = nodo;
42         } else {
43             y.right = nodo;
44         }
45
46         // Si el nuevo Nodo es una raíz, no hay necesidad de arreglos adicionales
47         if (Nodo.father == null){
48             nodo.color = 0;
49             return;
50         }
51
52         // Si el abuelo se vuelve rojo y el padre es rojo
53         if (Nodo.father != null && Nodo.father.father != null) {
54             arreglarInsert(Nodo);
55         }
56     }
57
58     // Corregir el árbol después de la inserción
59     private void arreglarInsert(Nodo k){
60         Nodo u;
61         while (k.father.color == 1) {
62             if (k.father == k.father.father.right) {
63                 // Si el padre es un hijo derecho
64                 u = k.father.father.left; // Tío
65                 if (u.color == 1) {
66                     // Caso 3.1
67                     u.color = 0;
68                     k.father.color = 0;
69                     k.father.father.color = 1;
70                     k = k.father.father;
71                 } else {
72                     if (k == k.father.left) {
73                         // Caso 3.2.2
74                         k = k.father;
75                         rotarDerecha(k);
76                     }
77                     // Caso 3.2.1
78                     k.father.color = 0;
79                     k.father.father.color = 1;
80                     rotarIzquierda(k.father.father);
81                 }
82             } else {
83                 // Si el padre es un hijo izquierdo
84                 u = k.father.father.right; // Tío
85
86                 if (u.color == 1) {
87                     // Caso 3.1
88                     u.color = 0;
89                     k.father.color = 0;
90                     k.father.father.color = 1;
91                     k = k.father.father;
92                 } else {
93                     if (k == k.father.right) {
94                         // Caso 3.2.2
95                         k = k.father;
96                         rotarIzquierda(k);
97                     }
98                 }
99             }
100         }
101     }
102 }

```

```

1 package ROJINEGRO;
2
3 public class Nodo {
4     public int data;
5     public Nodo father;
6     public Nodo left;
7     public Nodo right;
8     public int color; // 0 para negro, 1 para rojo
9 }

```

```

98         // Caso 3.2.1
99         k.father.color = 0;
100         k.father.father.color = 1;
101         rotarDerecha(k.father.father);
102     }
103 }
104     if (k == root) {
105         break;
106     }
107 }
108     root.color = 0;
109 }
110
111 // Rotación a la izquierda
112 private void rotarIzquierda(Nodo x) {
113     Nodo y = x.right;
114     x.right = y.left;
115     if (y.left != tnull) {
116         y.left.father = x;
117     }
118     y.father = x.father;
119     if (x.father == null) {
120         this.root = y;
121     } else if (x == x.father.left) {
122         x.father.left = y;
123     } else {
124         x.father.right = y;
125     }
126     y.left = x;
127     x.father = y;
128 }
129
130 // Rotación a la derecha
131 private void rotarDerecha(Nodo x) {
132     Nodo y = x.left;
133     x.left = y.right;
134     if (y.right != tnull) {
135         y.right.father = x;
136     }
137     y.father = x.father;
138     if (x.father == null) {
139         this.root = y;
140     } else if (x == x.father.right) {
141         x.father.right = y;
142     } else {
143         x.father.left = y;
144     }
145     y.right = x;
146     x.father = y;
147 }

```

MONTICULO BINARIO

Es similar a un árbol binario completo, pero tiene la propiedad que el nodo padre siempre es menor que sus hijos (en el caso de un min-montículo).

Un árbol binario completo es un árbol completamente lleno, con la excepción del nivel inferior, que debe llenarse de izquierda a derecha. La característica distintiva del árbol binario completo es que no faltan nodos en el árbol.

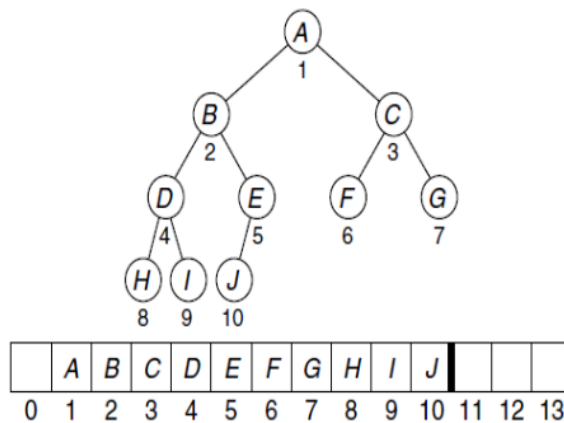
TIPOS DE MONTÍCULOS:

Min-Heap (Montículo Mínimo):

- El valor de cada nodo es **menor o igual** que sus hijos. PADRE < HIJO
- El menor está en la **raíz**.

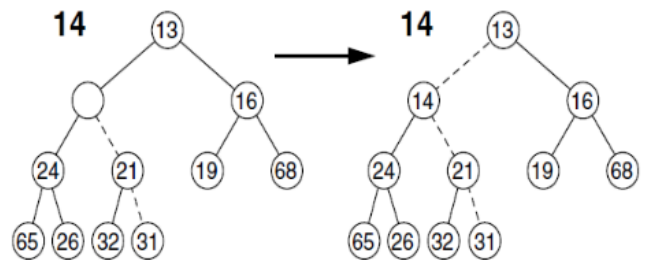
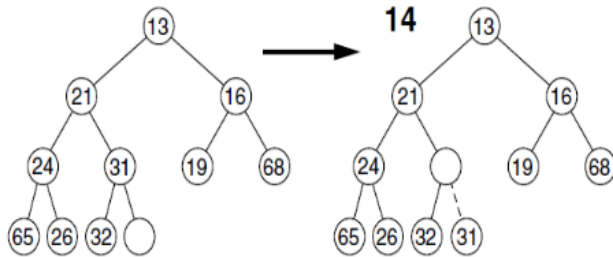
Max-Heap (Montículo Máximo):

- El valor de cada nodo es **mayor o igual** que sus hijos. PADRE > HIJO
- El mayor está en la **raíz**

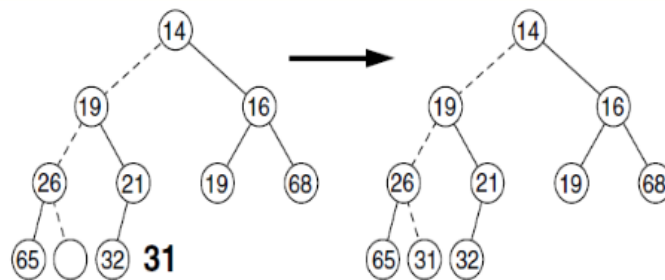
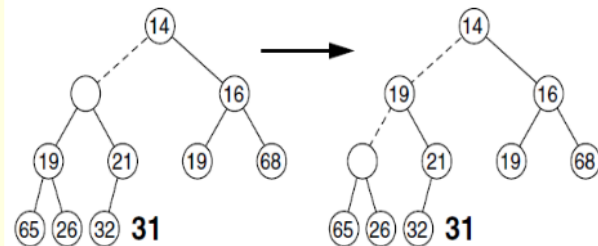
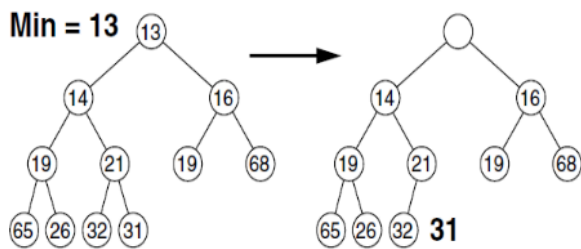


Dado un elemento situado en la posición i del arreglo, sabemos que su hijo izquierdo se encuentra en la posición $2i$ y el hijo derecho en la $2i+1$

Operaciones: INSERTAR



Operaciones: ELIMINARMIN



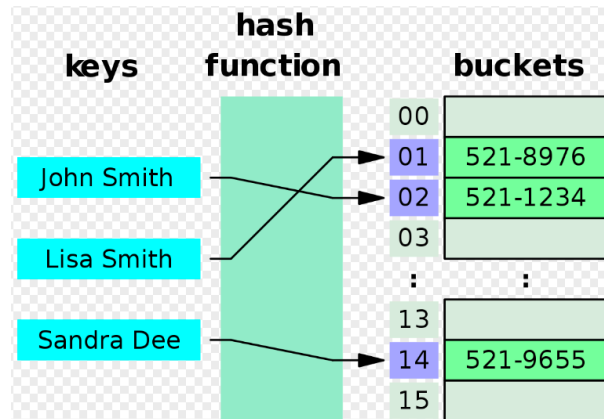
```

1 public class MonticuloBinario {
2     private static final int default_capacity = 100;
3     private int CurrentSize;
4     private Integer[] array;
5
6     // Constructor
7     public MonticuloBinario() {
8         CurrentSize = 0;
9         array = new Integer[default_capacity + 1]; // usamos índice desde 1
10    }
11
12    // Insertar un nuevo valor
13    public void insertar(Integer x) {
14        if (CurrentSize + 1 == array.length) {
15            duplicarArray();
16        }
17
18        CurrentSize++; // incrementamos el tamaño
19        int hueco = CurrentSize;
20        array[hueco] = x; // posición auxiliar para comparaciones
21
22        // Ajustar hacia arriba (subir el valor si es menor que sus padres)
23        while (hueco > 1 && comparar(x, array[hueco / 2]) < 0) {
24            array[hueco] = array[hueco / 2]; // mover el padre hacia abajo
25            hueco = hueco / 2; // subir el hueco a la posición del padre
26        }
27
28        array[hueco] = x; // colocamos el nuevo valor en su lugar final
29    }
30
31    // Comparar dos enteros, asegurando que no sean null
32    private int comparar(Integer a, Integer b) {
33        if (a == null || b == null) {
34            throw new IllegalArgumentException(s:"Los enteros no pueden ser nulos");
35        }
36        return a.compareTo(b); // comparación natural de Integer
37    }
38
39    // Duplicar el tamaño del array si se llena
40    private void duplicarArray() {
41        Integer[] nuevoArray = new Integer[array.length * 2];
42        for (int i = 1; i <= CurrentSize; i++) {
43            nuevoArray[i] = array[i];
44        }
45        array = nuevoArray;
46    }
47
48    // Eliminar el mínimo (la raíz)
49    public Integer eliminarMin() {
50        if (CurrentSize == 0) {
51            throw new IllegalStateException(s:"El montículo está vacío.");
52        }
53
54        Integer minimo = array[1]; // valor mínimo está en la raíz
55        array[1] = array[CurrentSize]; // mover el último elemento a la raíz
56        CurrentSize--; // reducir tamaño
57        percolateDown(hueco:1); // reordenar para mantener la propiedad de heap
58
59        return minimo;
60    }
61
62    // Reordenar hacia abajo (para mantener orden del heap después de eliminar)
63    private void percolateDown(int hueco) {
64        int hijo;
65        Integer temp = array[hueco];
66
67        while (hueco * 2 <= CurrentSize) {
68            hijo = hueco * 2; // hijo izquierdo
69
70            // Si hay hijo derecho y es menor que el izquierdo
71            if (hijo + 1 <= CurrentSize && comparar(array[hijo + 1], array[hijo]) < 0) {
72                hijo = hijo + 1; // ahora es el hijo derecho
73            }
74
75            // Si el hijo menor es menor que temp, bajar ese hijo
76            if (comparar(array[hijo], temp) < 0) {
77                array[hueco] = array[hijo]; // mover el hijo hacia arriba
78                hueco = hijo; // bajar el hueco
79            } else {
80                break; // ya está en el lugar correcto
81            }
82        }
83
84        array[hueco] = temp; // colocar el valor original en su nueva posición
85    }
86
87    }
88
89    }
90
91    }
92
93    }
94

```

TABLA HASH

Es una estructura de datos que asocia llaves o claves con valores. La operación principal que soporta de manera eficiente es la búsqueda: permite el acceso a los elementos (teléfono y dirección, por ejemplo) almacenados a partir de una clave generada usando el nombre, número de cuenta o id. Funciona transformando la clave con una función hash en



un hash, un número que la tabla hash utiliza para localizar el valor deseado.

El empleo de hash introduce una complicación: es posible que a dos elementos distintos les corresponda el mismo índice, cuando esto sucede, se produce una colisión, los métodos más simples para resolverlo son:

- **Exploración lineal:** Si el primer índice está ocupado, se intenta el siguiente, y así sucesivamente, hasta encontrar un espacio vacío o hasta que se recorra toda la tabla.
- **Exploración cuadrática:** se exploran posiciones a intervalos cuadráticos. Por ejemplo, si la colisión ocurre en el índice i , se probarían los índices $i + 1^2$, $i + 2^2$, $i + 3^2$, y así sucesivamente.
- **Encadenamiento separado:** Consiste en crear listas o estructuras de datos (como listas enlazadas) en cada índice de la tabla hash. Cuando ocurre una colisión, el nuevo elemento se agrega a la lista correspondiente al índice, evitando así que se sobrescriban datos.

```

1 public class TablaHash {
2
3     private Nodo[] buckets; // Array de buckets (cada uno es una lista enlazada)
4     private int numBuckets; // Cantidad de buckets en la tabla
5     private int size; // Cantidad total de elementos almacenados
6
7     // Constructor
8     public TablaHash() {
9         numBuckets = 10; // Fijamos tamaño inicial de la tabla
10        buckets = new Nodo[numBuckets]; // Inicializamos array de buckets
11        size = 0; // Empezamos con tabla vacía
12    }
13
14    // =====
15    // FUNCION HASH PARA CLAVES TIPO STRING
16    // =====
17    private int hash(String key) {
18        int hashVal = 0;
19        for (int i = 0; i < key.length(); i++) {
20            // Usamos base 128 (ASCII) y módulo del tamaño de la tabla
21            hashVal = (hashVal * 128 + key.charAt(i)) % numBuckets;
22        }
23        return hashVal;
24    }
25
26    // =====
27    // INSERTAR ELEMENTO EN LA TABLA
28    // =====
29    public void put(String key, String value) {
30        int index = hash(key); // Calcula índice usando la función hash
31        Nodo newNode = new Nodo(key, value); // Crea nuevo nodo
32
33        if (buckets[index] == null) {
34            // Si el bucket está vacío, lo asignamos directamente
35            buckets[index] = newNode;
36        } else {
37            // Si ya hay elementos, agregamos al final de la lista (encadenamiento)
38            Nodo current = buckets[index];
39            while (current.next != null) {
40                current = current.next;
41            }
42            current.next = newNode;
43        }
44
45        size++; // Incrementamos cantidad de elementos
46    }
47
48    // =====
49    // BUSCAR VALOR POR SU CLAVE
50    // =====
51    public String search(String key) {
52        int index = hash(key); // Calculamos índice
53        Nodo current = buckets[index]; // Vamos al bucket correspondiente
54
55        while (current != null) {
56            if (current.key.equals(key)) {
57                return current.value; // Si la clave coincide, retornamos el valor
58            }
59            current = current.next; // Avanzamos al siguiente nodo
60        }
61
62        return null; // No se encontró la clave
63    }
64
65    // =====
66    // ELIMINAR ELEMENTO POR SU CLAVE
67    // =====
68    public void remove(String key) {
69        int index = hash(key); // Calculamos índice
70        Nodo current = buckets[index]; // Nodo actual
71        Nodo previous = null; // Nodo anterior para relinko
72
73        while (current != null) {
74            if (current.key.equals(key)) {
75                if (previous == null) {
76                    // Si está al inicio de la lista
77                    buckets[index] = current.next;
78                } else {
79                    // Si está en el medio o al final
80                    previous.next = current.next;
81                }
82
83                size--; // Actualizamos tamaño
84                System.out.println("Elemento con clave " + key + " eliminado.");
85                return;
86            }
87
88            previous = current;
89            current = current.next;
90        }
91
92        // No se encontró
93        System.out.println("Elemento con clave " + key + " no encontrado.");
94    }
95 }

```

```

1 public class Nodo {
2     String key;
3     String value;
4     Nodo next;
5
6     public Nodo(String key, String value) {
7         this.key = key;
8         this.value = value;
9         this.next = null;
10    }
11 }

```